

# ハコモノ SaaS

2025年6月26日 11:51

**SaaSの定義**

SaaSとは、顧客とプロバイダーの両方に最大の価値をもたらす、摩擦の少ないサービス中心のモデルでソリューションを提供できるビジネスとソフトウェアのデリバリーモデルです。

成長、市場拡大、イノベーションを促進するビジネス戦略の柱として、俊敏性と運用効率に重点を置いています。

出典: ゴールディング・トッド 河原 哲也、松本 匠人 マルチテナントSaaSアーキテクチャの構築、第1章, pp.1-25 東京: オライリー・ジャパン, 2025. PJ kasanaka

俊敏性と運用効率

→競合がいつ現れるかわからない

**SaaSにおけるアーキテクチャが目指す俊敏性、運用効率**

**俊敏性**

- 市場や顧客の変化に対し、開発 → テスト → デプロイといったサイクルを迅速に繰り返し、機能追加や改善を迅速に実現できること

**運用効率**

- 急速に成長するビジネスを運用面から継続的かつ安定的に支えられる技術基盤・プラットフォームであること

→ アーキテクチャとして上記を満たせる構成を追い求めることが必要

PJ kasanaka

フェーズ

顧客が少ないケースだとサイロ化モデル

**クラウドにおけるSaaS化のフェーズ**

フェーズ0. クラウドリフト  
クラウド上でのサーバー移植

フェーズ1. シングルテナント移行  
サイロ化モデル

フェーズ2. 一部マルチテナント移行  
データ層共通化

フェーズ3. 完全マルチテナント移行  
アプリケーション層共通化

フェーズ4. クラウド最適化  
サーバーレス化

ユーザ数が少ない時は運用可能

ユーザ数が増えると管理の最適化が必要

参照: SaaS on AWS を成功に導くためのポイントとは？ 第1回  
<https://www.amazon.com/gp/builders-flash/202116/s/sas-on-aws/> PJ kasanaka

テナント分離

**SaaSにおけるテナント分離**

SaaSにおいて重要な"テナント分離"

| サイロモデル                                         | プールモデル                                             | ブリッジモデル                                                         |
|------------------------------------------------|----------------------------------------------------|-----------------------------------------------------------------|
| テナントごとに完全に独立したリソースを割り当て他のテナントと一切共有しないアーキテクチャ形式 | 全テナントが同一のコンピューティングリソース、データストアを共有し、一括管理するアーキテクチャ形式。 | フロント(ウェブ層)は全テナント共通のプール、バック(アプリ層)はテナント単位またはグループ単位で分離するアーキテクチャ形式。 |

PJ kasanaka

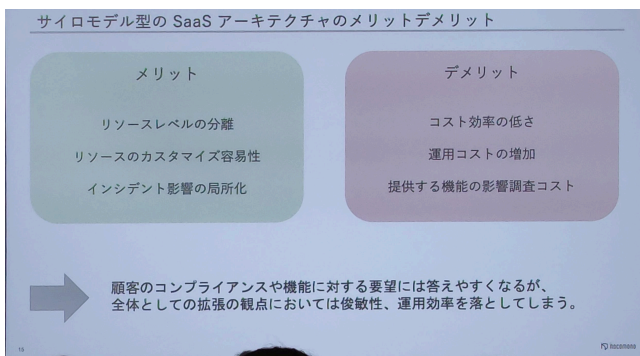
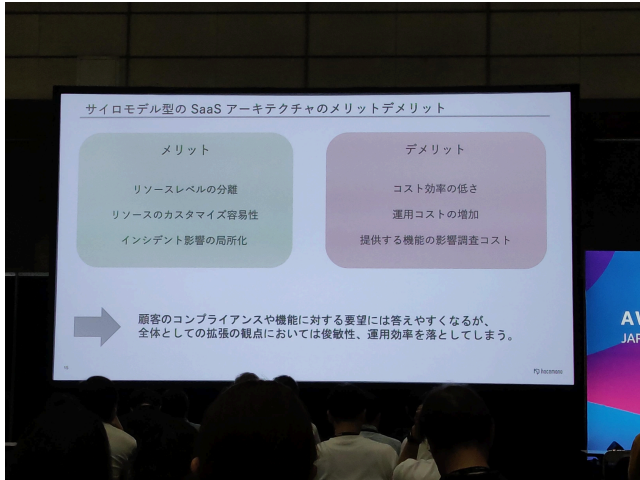
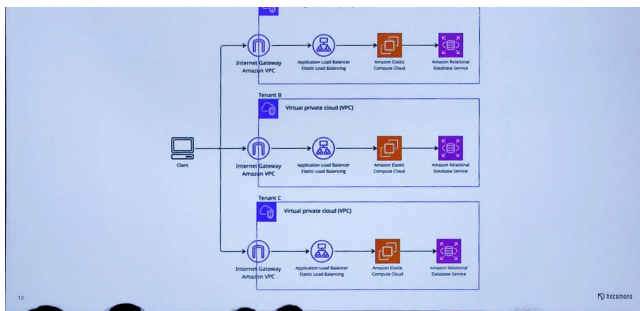
最初はテナントごとに明確に分離

受託開発のノウハウから始めた

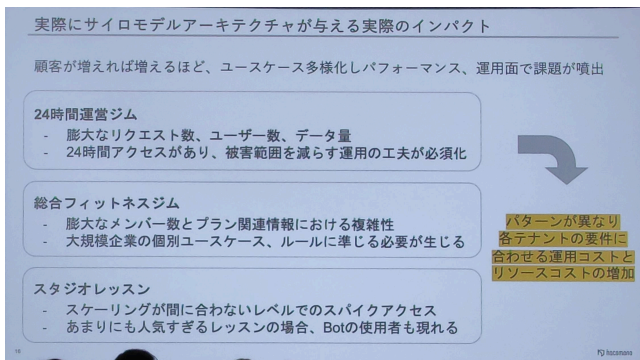
個別要件があった

**当初のアーキテクチャ(サイロモデル)**

Virtual private cloud (VPC)

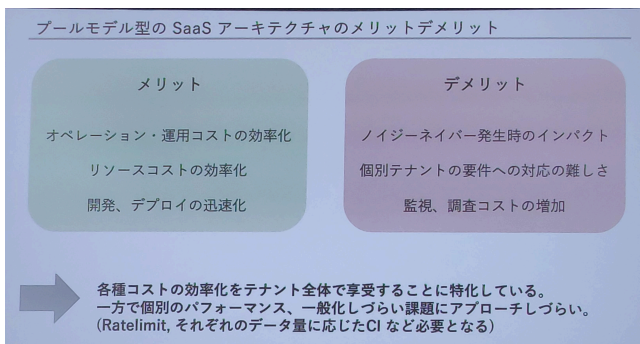


### 顧客が増えるとユースケースが多様化



Botでの使用者って・・・

### プール型



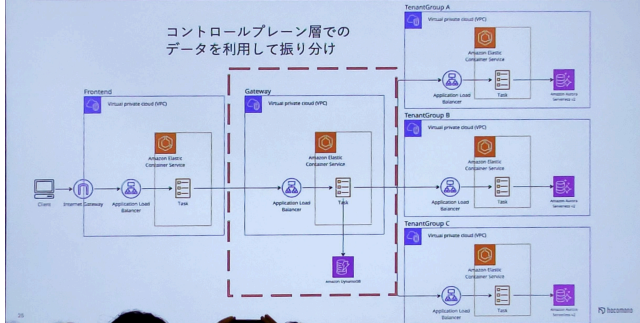


### 一方で、どんなSaaSでもプールモデルが最強のモデルなのか？

- SaaSと言えばコスト効率、運用効率にメリットが生じるプールモデルのアーキテクチャのイメージが定着しており、SaaSの定義からも理にかなっている。
- しかしながらSaaSのアーキテクチャには正解がない所以として、業界・ドメインの特徴や自社プロダクト、サービスの特徴次第では合わない場合も出てくることもある。  
e.g 一部テナントではコンプライアンス準拠が重視される、お客様ごとにユースケースが異なっている など
- したがって、対象としたドメインのニーズを満たしつつ、プロダクトの特性やフェーズを理解して迅速性と運用効率性を満たすアーキテクチャを見出す必要性が出てくる。

### Web層とバックエンドのアーキテクチャを分離

#### ブリッジモデルの考えを取り入れたアーキテクチャ

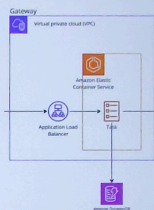


### コントロールプレーン

#### ブリッジモデルへの移行作業の準備(コントロールプレーン)

##### コントロールプレーン

- 旧基盤、新基盤の移行の過渡期を支える、所属するテナントへのリクエストを正確に振り分ける機構も構築。
- Amazon DynamoDBにはテナントのID関連のデータが保存されており、このコントロールプレーンのデータに基づいて振り分ける。
- Amazon DynamoDBのデータにはどのテナントグループにテナントが所属しているかについても記録。オンボーディング時、リソース構築時にここに情報を蓄積。



### なぜブリッジモデルにしたか？

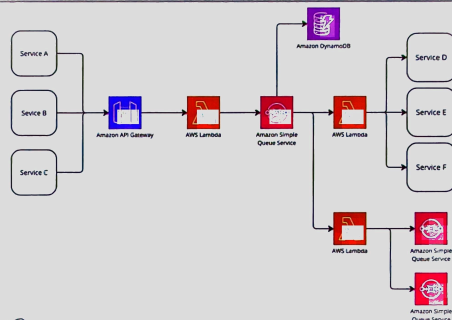
- 当時のアプリケーションの問題となりうる箇所を分析。  
=> DB関連の課題が多く、ノイズネイバーにつながる可能性があった。
- 全テナントがまとめられたリソースで全テナントにインパクトを与えないようにインシデントの影響を局所化しつつ改善を進める方針とし、一定数のテナントでまとめる方針に。  
=> 結果的に cell-based architectureと近い設計思想になっていたことをAWS re:Invent 2024で知る。
- 結果的に、プールモデルのメリットをできる限り享受しつつ、自社プロダクトの独自のリスクを極小化。

### コスト観点でのメリット

- インフラコストがサイロモデルと比べ、約2000万/月 程度削減。  
Amazon Relational Database Service (Amazon RDS) をテナントグループごとに分割してもこの威力。
- 運用管理コストに関しては、特にミドルウェアのアップデートやスケーリング対応時に軽減されたことを実感。
- リソースパフォーマンスにおいてもオーバーヘッドがサイロモデルの際よりも少なくなり、適切なリソースパフォーマンスを提供できるように。

### メッセージング

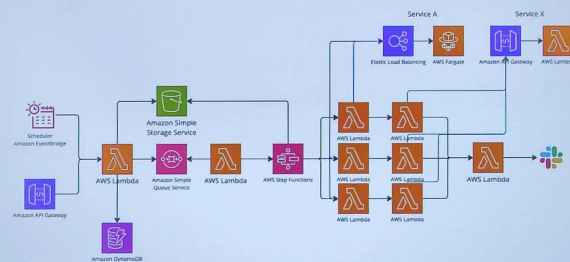
## マルチプロダクトに向けたイベントメッセージングシステム



## マルチプロダクトに向けたイベントメッセージングシステム

- イベントバスとしてスキーマ定義されているイベントのみを配信する仕組みであるため、fire-and-forgetの方式を順守し、マルチプロダクト利用を想定。
- 想定 of Deliver (送信側) と Reciever (受信側) が正確にイベントを送受信できるような仕組みを実現。JWT による改竄検知の方式を導入。
- protovalidateを使用したschema 判別機構を導入し、開発継続後も互換性を保ったスキーマ開発を考慮。
- アプリケーションで受け取る場合において Deliver と Reciever で利用できるライブラリを開発・提供。

## Sagaパターン形式のジョブ処理システム



## Sagaパターン形式のジョブ処理システム

- プロダクトが内部で処理していたジョブ実行を、外部リソース利用のジョブ実行に切り替えることでコンテナ内部のリソース使用量を減らしノイズネイバー軽減へ。
- インフラコスト観点ではRuntaskを起因とした AWS Fargateタスクでのジョブ処理をしていたが、起動しなくてもアーキテクチャにすることで費用軽減。
- AWS Step Functions によるリトライ機構によるお客様対応スピードと信頼性を向上。
- エラーになった場合のログフォーマットやアラートフローを統一し調査・運用コストを軽減。

## セキュリティ

## アカウント戦略と権限付与体制、セキュリティ

- 組織が大きくなるにつれて開発人員が増え、アカウント体制や権限付与を効率的に拡張する必要が出てくる。  
=> AWS IAM Identity Center, AWS Config, AWS Organizations, AWS Control Tower
- サービスが大きくなるにつれ、外部からの脅威に対する備えが必要となってくる。  
=> Amazon GuardDuty, AWS Security Hub, AWS CloudTrail



## 将来像

## さらに迅速性、効率性を追い求めていくために



